Java es uno de los lenguajes más utilizados para la programación en todo el mundo. Un lenguaje de programación no es más que un conjunto de reglas para que los programadores puedan escribir instrucciones que nos permite comunicarnos con la computadora.

¿Qué es un algoritmo?

Un algoritmo es un conjunto de instrucciones que se realizan para resolver un problema. El algoritmo debe especificar las acciones que se van a ejecutar, (que deben ser finitas) y en qué orden. La programación se encarga de resolver problemas a través de los algoritmos.
Un algoritmo recibe información de entrada, la procesa y genera una salida.

Ejemplo: Elaboración de una tortilla de patatas.
Datos de entrada: Huevos, aceite, patatas, sartén, fuego
Datos de salida: Tortilla de patatas

Algoritmo:
1. Poner el aceite en la sartén.
2. Poner la sartén al fuego.
3. Cuando el aceite esté caliente, freir las patatas.
4. Batir los huevos y añadir las patatas
5. Añadir los huevos a la sartén y dale vueltas.

Otros ejemplos: Determinar si un numero es par, ordenar alfabéticamente un conjunto de palabras, calcular el producto de dos números, etc...

¿Qué es un programa?

Los programas son algoritmos escritos con sentencias específicas de un lenguaje de programación concreto para que un ordenador pueda procesarlo. Dependiendo de la complejidad del programa puede estar compuesto por un algoritmo (un único conjunto de instrucciones) o por muchos que resuelven diferentes tareas.

¿Cuáles son los pasos que seguimos para escribir un programa?

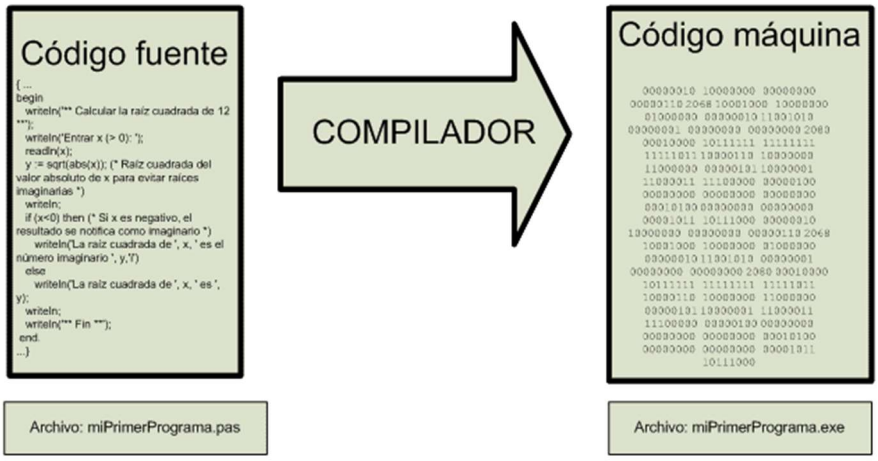
1. Primero partimos de un problema que es lo que tenemos que resolver.
2. A continuación, hacemos un análisis de ese problema. En el análisis, entre otras cosas, debemos:
    a. Entender bien el problema, sabiendo concretamente qué hay que resolver
    b. Revisar de qué recursos disponemos para resolverlo
    c. Intentar dividir el problema en otros más pequeños (divide y vencerás)
3. Tras el análisis, procedemos a hacer el diseño de la solución. Consiste en definir nuestro algoritmo. Se suele utilizar algún método que sea independiente del lenguaje de programación utilizado. Existen distintas técnicas, pero podemos destacar:
    a. Diagramas de flujo: utilizan símbolos gráficos y a través de lineas de flujo se indica la secuencia en que se deben ejecutar.
    b. Pseudocódigo: notación en lenguaje natural
4. Una vez hecho el diseño, transcribiremos las instrucciones que hemos diseñado al lenguaje de programación elegido. Es decir, codificaremos, escribiremos el programa en código.
5. Terminado de escribir el código, el último paso será probar que efectivamente resuelve el problema. Es importante probar muy bien nuestro programa. Siempre hay errores ocultos. Si se encuentran errores, dependiendo de su gravedad, es posible que sea necesario rehacer el código, el diseño o incluso el análisis.

¿Qué lenguajes de programación podemos utilizar?

Hay muchos lenguajes de programación. Nosotros vamos a aprender en este curso a utilizar JAVA. Alguna de sus características:

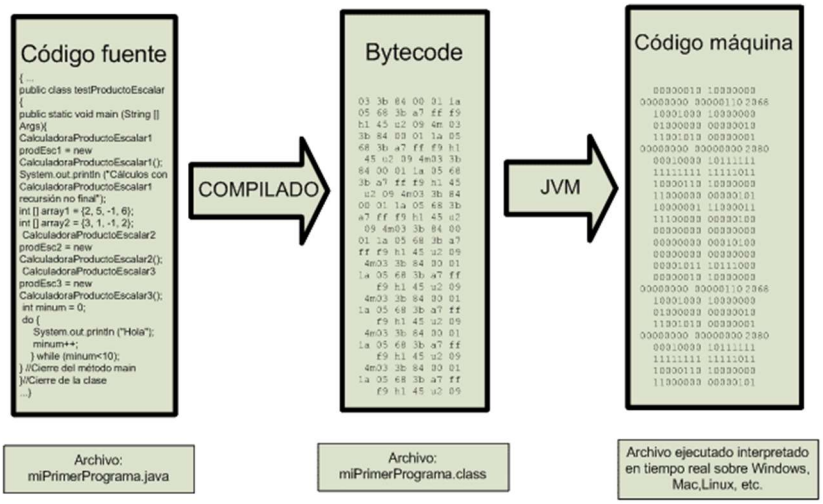
- Es uno de los lenguajes más utilizados en el mundo en diferentes ámbitos tecnológicos (programación web, aplicaciones de escritorio, etc)
- Lenguaje orientado a objetos
- Es un lenguaje **compilado** (el código escrito necesita ser compilado a lenguaje máquina para poder ser ejecutado)

La **compilación** es un proceso fundamental en la creación de software que traduce el código escrito por los programadores en un lenguaje de alto nivel (como Java) a un lenguaje de bajo nivel que la máquina pueda entender y ejecutar, conocido como **código máquina** o **código objeto.**

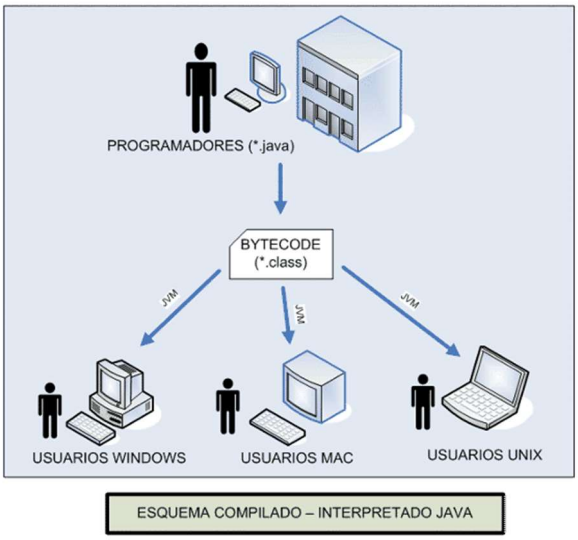


Sin embargo, en este gráfico, el archivo ejecutable no vale para cualquier plataforma. Si hemos generado el ejecutable en Windows, no se podrá usar en Linux. En Java, el proceso no se corresponde con el esquema anterior. **Java** se diferencia notablemente de otros lenguajes de programación debido a su enfoque en la **portabilidad** y la **independencia de la plataforma**.

¿Cómo? Creando un paso intermedio. De esta forma, los programas en java no se ejecutan directamente en nuestro ordenador sino que Java simula una máquina virtual con su sistema operativo. Es decir, del código *fuente* (código escrito por el programador), pasamos a un código intermedio llamado "*bytecode*". Este código es entendible por la máquina virtual Java y será quien ejecute el programa:



De esta forma, Java, se puede ejecutar en cualquier máquina independientemente del sistema operativo, porque en realidad se ejecuta en su propia máquina virtual, que se instala cuando instalamos Java.



ESQUEMA COMPILADO – INTERPRETADO JAVA

En Java escribimos archivos con extensión **.java.** Cuando compilamos, se genera un archivo **.class,** escrito en un lenguaje de máquina virtual (bytecode) e interpretable por la máquina virtual. Para que nuestro ordenador ejecute el programa, hay que interpretar o traducir el archivo .class a un código en lenguaje de maquina real, que lo hará el intérprete Java de la maquina virtual.

Objetos y Clases

Java es un lenguaje orientado a objetos y es necesario conocer algunos términos fundamentales como objetos y clases.
Un **objeto** en Java es lo mismo que un objeto en el mundo real. No podemos tocarlo, pero sí existe como 'algo' que contiene información sobre sí mismo y con el que puedes interactuar.

Ejemplo: Piensa en tú teléfono móvil. Tiene un color, una marca, y supongamos que está en silencio.
Piensa ahora en otro móvil. Puede que tenga otro color , otra marca y que esté apagado.
Pues bien, si quisiéramos crear un objeto de tipo Teléfono en Java, primero debemos crear "un molde" con las características (color, marca …) así como el comportamiento del objeto (apagar, encender el móvil).
Utilizando este molde podremos crear tantos objetos de tipo Teléfono como necesitemos. A este molde se le denomina **clase** en Java.

En el ejemplo, tendríamos la clase Telefono, con:

- *Propiedades*: color, marca y modelo
- *Operaciones*: apagar, encender teléfono

Y podremos tener varios teléfonos, definiendo todos los **objetos** que necesitemos que pertenezcan a dicha clase:

Teléfono 1: marca Samsung, color negro
Teléfono 2: marca Apple y color azul
Teléfono 3: marca Google y color gris

Diremos que un objeto es una instancia de una clase. Es decir, teléfono 1 es una instancia de la clase Telefono.

A partir de este ejemplo podemos definir:

Objeto: entidad existente en la memoria del ordenador que tiene unas propiedades y unas operaciones disponibles específicas (métodos).

Clase: plantilla o abstracción que define un tipo de objeto especificando qué propiedades (atributos) y operaciones disponibles va a tener.

¿Cómo escribimos nuestro primer programa en Java?

Utilizaremos Eclipse IDE para programar (ver diapositivas que explican como descargarlo y utilizarlo). Para crear un programa que podamos ejecutar, crearemos una clase con el nombre que queramos. Dentro de esa clase, creamos un método **main**. Ahí dentro podemos poner nuestras instrucciones.

```
public class MiPrograma {

    public static void main(String[] args) {
        // aquí irían mis instrucciones

    }

}
```

¿Qué es una variable?

Una variable me permite almacenar datos de diferente tipo. Para declarar una variable basta con escribir el tipo de dato y el nombre de la variable.

```
Tipo nombreVariable;
```

¿Qué tipos de datos podemos usar?

Muchos tipos de datos dentro del paradigma de la orientación a objetos. Pero esto lo veremos en próximos temas. Por ahora, los datos básicos que vamos a manejar son estos:

- Texto (cadena): String
- Números enteros cortos: Integer o int
- Números enteros largos: Long o long
- Números decimales: Float o float
- Números decimales largos: Double o double

Por ejemplo, para declarar una variable de tipo Texto con nombre “mensaje” lo haríamos así:

```
String mensaje;
```

Aunque los iremos viendo poco a poco, estos son los tipos que se pueden usar en Java.

		NOMBRE	TIPO	OCUPA	RANGO APROXIMADO
TIPOS DE DATOS EN JAVA	TIPOS PRIMITIVOS (sin métodos; no son objetos; no necesitan una invocación para ser creados)	byte	Entero	1 byte	-128 a 127
		short	Entero	2 bytes	-32768 a 32767
		int	Entero	4 bytes	$2*10^9$
		long	Entero	8 bytes	Muy grande
		float	Decimal simple	4 bytes	Muy grande
		double	Decimal doble	8 bytes	Muy grande
		char	Carácter simple	2 bytes	---
		boolean	Valor true o false	1 byte	---
	TIPOS OBJETO (con métodos, necesitan una invocación para ser creados)	Tipos de la biblioteca estándar de Java	String (cadenas de texto) Muchos otros (p.ej. Scanner, TreeSet, ArrayList...)		
		Tipos definidos por el programador / usuario	Cualquiera que se nos ocurra, por ejemplo Taxi, Autobus, Tranvia		
		arrays	Serie de elementos o formación tipo vector o matriz. Lo consideraremos un objeto especial que carece de métodos.		
		Tipos envoltorio o wrapper (Equivalentes a los tipos primitivos pero como objetos.)	Byte		
			Short		
			Integer		
			Long		
			Float		
			Double		
			Character		
			Boolean		

¿Cómo le damos valor a una variable?

Basta con poner el nombre de la variable, luego el operador igual (=) y a continuación el valor que le queremos dar. Este valor puede ser un valor directo, u otra variable, o una operación entre variables/valores.

El valor de una variable puede cambiar cuando queramos.

Ejemplos:

```
public static void main(String[] args) {
    String mensaje;
    mensaje = "Hola";
    mensaje = "Hola " + "qué tal";

    Integer numero1;
    numero1 = 84;
    Integer numero2;
    numero2 = 65 + numero1;
}
```

También podemos darle valor a una variable a la vez que la declaramos, en la misma línea:

```
String mensaje = "Hola mundo";
```

¿Qué es una constante?

Una constante funciona igual que una variable, pero el primer valor que recibe cuando la declaramos ya no puede cambiar. Se utilizan para registrar valores fijos.

Se debe declarar fuera del método main arriba del todo. Se hace de este modo:

```
public static final Tipo NOMBRE_CONSTANTE = valor;
```

Se utilizan igual que las variables.

Ejemplos de dos constantes de tipo String y Integer

```
public class MiPrograma {

    public static final Integer MESES_DEL_AÑO = 12;
    public static final String SALUDO = "hola";

    public static void main(String[] args) {
        System.out.println(SALUDO + " los meses del año son " + MESES_DEL_AÑO);
    }

}
```

¿Cómo podemos hacer conversiones de un tipo de dato a otro?

De String	**A Integer:**	Integer.valueOf("12")
	A Long:	Long.valueOf("12")
	A Float:	Float.valueOf("12.34")
	A Double:	Double.valueOf ("12.34")
De cualquier tipo	**A String:**	numero.toString()
De otro tipo distinto a String	**A otro tipo distinto a String:**	numero.intValue();
		numero.longValue();
		numero.floatValue();
		numero.doubleValue();

Cuidado, algunos pueden generar un error si no se puede hacer la conversión correctamente.
Ejemplos:

```
String cadena = "123";

// De String a número
Integer numeroInt = Integer.valueOf(cadena);
Long numeroLong = Long.valueOf(cadena);
Float numeroFloat = Float.valueOf(cadena);
Double numeroDouble = Double.valueOf(cadena);

// De número a String
cadena = numeroInt.toString();
cadena = numeroLong.toString();
cadena = numeroFloat.toString();
cadena = numeroDouble.toString();

// De un tipo de número a otro tipo de número
numeroInt = numeroLong.intValue();
numeroLong = numeroInt.longValue();

numeroInt = numeroDouble.intValue();
numeroDouble = numeroInt.doubleValue();
```

¿Cómo mostramos algo por consola?

Para imprimir mensajes por consola utilizamos esta instrucción:

```
System.out.println();
```

Entre paréntesis tenemos que indicar qué queremos imprimir. Puede ser una cadena, o una variable de tipo String, o varias variables/valores que concatenados sean una cadena.

Ejemplo:

```
public static void main(String[] args) {
    String mensaje = "Hola mundo";
    System.out.println(mensaje);
    System.out.println("Hola qué tal");
    System.out.println(mensaje + "!!");
}
```

¿Cómo obtenemos algo que el usuario escriba por teclado?

En nuestro código podemos utilizar la clase proporcionada por el profesor UserReader. Esta clase tiene un método llamado read() que nos devuelve un String con lo que haya escrito el usuario:

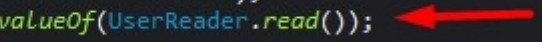
```
System.out.println("Dime una palabra");
String palabra = UserReader.read();
```

Si queremos leer un número tenemos que hacer una conversión de tipo:

```
System.out.println("Dime un número entero");
String mensaje = UserReader.read();
Integer numero = Integer.valueOf(mensaje);
```

Lo podemos poner todo en la misma línea:

```
System.out.println("Dime un número entero");
Integer numero = Integer.valueOf(UserReader.read());
```



¿Cómo podemos incluir en una cadena un carácter especial?

En las cadenas (String) no podemos incluir un salto de línea o unas dobles comillas como sin nada. Se tratan de caracteres especiales. Para incluirlos se deben escribir de este modo:

Secuencia de escape	Valor
\b	Retroceso o *backspace* (equivalente a \u0008)
\t	Tabulador (equivalente a \u0009)
\n	Nueva línea (equivalente a \u000A)
\f	Salto de página (equivalente a \u000C)
\r	Retorno de carro (equivalente a \u000D)
\"	Doble comilla (equivalente a \u0022)
\'	Comilla simple (equivalente a \u0027)

¿Cómo podemos escribir comentarios en el código?

Los comentarios en el código no se ejecutan, se ignoran. Nos sirven para aclarar qué es lo que estamos programando, para que en el futuro sepamos qué significa cada instrucción. O para explicar por qué hemos tomado determinadas decisiones.

Hay dos formas de escribir comentarios:

- En una línea: basta con poner "//" antes del comentario
- En varias líneas: Hay que empezar con "/*" y terminar con "*/"

Ejemplos:

```
public static void main(String[] args) {
    // Comentario de una sólo línea

    /* Comentario
     de muchas
     líneas */
}
```

¿Cómo debemos escribir los nombres de variables, paquetes y clases?

Hay una serie de normal a la hora de escoger nombres a las cosas que vamos creando cuando programamos. Algunas normas son obligatorias. Otras son por convención. Ambas deben seguirse:

- Nombres de paquetes: todo en minúscula y sin espacios. No pueden empezar por números. Ejemplo: nombredepaquete
- Nombres de clases: La primera letra SIEMPRE en mayúscula. Los espacios se indican con camelCase. No pueden empezar por números. Ejemplo: NombreDeClase
- Nombres de variables: La primera letra SIEMPRE en minúscula. Los espacios se indican con camelCase. Ejemplo: nombreDeVariable
- Nombres de constantes: Todas las letras en mayúscula. Los espacios se indican con guiones bajos. Ejemplo: NOMBRE_DE_CONSTANTE

AVANZADO (No es necesario conocer para aprobar)

La clase UserReader funciona dentro con Scanner. Scanner es una clase especial de Java que debemos inicializar en una variable en primer lugar. Luego lo podemos utilizar todas las veces que queramos. Cada vez que usamos el Scanner para leer un dato por el teclado, la ejecución del programa se pausa hasta que el usuario escribe.

```
public static void main(String[] args) {
    // Inicializamos Scanner en una variable
    Scanner scanner = new Scanner(System.in);

    // Mostramos mensaje al usuario
    System.out.println("Indica un texto por favor");

    // Lo utilizamos
    String mensaje = scanner.nextLine();

    // Mostramos mensaje al usuario
    System.out.println("Indica un número por favor");

    // Volvemos a utilizar el scanner
    Integer numero = scanner.nextInt();

    // Mostramos lo que hemos leído
    System.out.println("He leido esto: " + mensaje + " y esto " + numero);
}
```

Cuando declaramos el Scanner es necesario importar la clase Scanner. Eclipse nos lo hace automáticamente añadiendo este código al principio de nuestra clase:

```
package miprograma;

import java.util.Scanner;

public class MiPrograma {
```